

Module 8 – Web Services, API, Extensions

1 Objective: Understand the concept and importance of payment gateways in e-commerce?

1. What is a Payment Gateway?

A **payment gateway** is a technology that allows customers to make online payments on an e-commerce website using:

- Debit/Credit cards
- UPI
- Net banking
- Digital wallets (Paytm, PhonePe, etc.)

It acts as a **bridge between the customer, the online store, and the bank** to securely process transactions.

How a Payment Gateway Works (Simple Flow)

1. Customer selects a product and clicks **Pay Now**
2. Enters payment details
3. Payment gateway **encrypts** the data
4. Bank checks if funds are available
5. Payment is approved or declined
6. Result is sent back to the website
7. Order is confirmed

Why Payment Gateways Are Important in E-Commerce

1. Security

- Protects sensitive data using encryption
- Prevents fraud and unauthorized access
- Follows standards like **PCI-DSS**

2. Fast & Convenient Payments

- Instant transaction processing
- Supports multiple payment options
- Improves customer experience

3. Global Reach

- Accept payments from customers worldwide
- Supports different currencies and countries

4. Increases Sales

- Easy checkout = fewer abandoned carts
- Customers trust secure payment systems

5. Automatic Record Keeping

- Stores transaction history
- Helps with accounting and reporting

Q1 Explain the role of payment gateways in online transactions

1. Collects Payment Information

- Takes card/UPI/net banking details from the customer
- Sends the data securely for processing

2. Encrypts Sensitive Data

- Converts payment details into encrypted form
- Protects against hacking and data theft

3. Authorizes the Payment

- Communicates with the customer's bank
- Checks if sufficient balance is available
- Approves or declines the transaction

4. Transfers Money Securely

- Moves funds from the customer's bank to the merchant's account
- Ensures correct settlement

5. Confirms Transaction Status

- Sends success or failure message to both customer and merchant
- Helps generate order confirmation

6. Prevents Fraud

- Uses tools like OTP, 3D secure, CVV verification
- Detects suspicious transactions

7. Supports Multiple Payment Methods

- Debit/Credit cards
- UPI
- Wallets
- Net banking

8. Maintains Transaction Records

- Stores payment history
- Helps businesses with accounting and dispute resolution

Q2 Discuss the security measures involved in payment gateway integration.

Security Measures in Payment Gateway Integration

Payment gateway integration requires strong security to protect customer data, prevent fraud, and ensure safe online transactions. The following are the key security measures used:

1. Data Encryption (SSL/TLS)

- Converts sensitive data (card number, CVV, UPI) into unreadable code
- Uses **HTTPS / SSL certificates**
- Prevents hackers from reading data during transmission

2. PCI-DSS Compliance

- International security standard for handling card data
- Ensures:
 - Secure storage
 - Restricted access
 - Regular security testing

3. Tokenization

- Replaces actual card details with a **unique token**
- Real data is never stored on the merchant's server
- Reduces risk of data theft

4. Two-Factor Authentication (2FA / OTP)

- Requires OTP or biometric verification
- Adds an extra layer of identity check
- Common for cards and UPI transactions

5. CVV & Address Verification (AVS)

- CVV confirms the user physically owns the card
- AVS verifies billing address (mostly international)

6. Fraud Detection Systems

- Uses AI & rules to detect:
 - Unusual locations
 - High-value transactions
 - Repeated failed attempts
- Automatically blocks suspicious activity

7. Secure APIs

- Uses authenticated and encrypted APIs
- Prevents unauthorized access to payment systems

8. Regular Security Audits & Monitoring

- Continuous system monitoring
- Penetration testing
- Log tracking to detect threats

9. Firewall & DDoS Protection

- Protects servers from cyber-attacks
- Blocks malicious traffic

10. Limited Data Storage

- Merchants avoid storing card details
- Only necessary transaction references are saved

Q3 Compare and contrast different payment gateway options (e.g., PayPal, Stripe, Razor pay)

Feature	PayPal	Stripe	Razorpay
Best for	Global online payments & freelancers	Developers, SaaS, global businesses	India-focused e-commerce & UPI domination
Primary Strength	Trusted brand & buyer protection	Highly customizable APIs	Strong support for Indian payment modes
International Support	Excellent (200+ markets)	Excellent (100+ currencies)	Available but limited vs Stripe/PayPal
Domestic Payments (India)	Limited/Only cross-border	Available but less local support	Excellent (UPI, cards, wallets, net banking)
Transaction Fees (approx)	~2.9–4% + fixed fee & FX markup (digitalpaymentguru.com)	~2.9% + ₹30 (domestic), ~3.9%+ FX (international) (digitalpaymentguru.com)	~2% domestic, UPI often 0%, ~3–4% international (digitalpaymentguru.com)
Settlement Time	1–3 days typically (digitalpaymentguru.com)	2–7 days standard (digitalpaymentguru.com)	T+2 (can be T+1/instant for fees) (thefinrate.com)
Ease of Integration	Simple buttons, plugins (thefinrate.com)	Excellent APIs, highly flexible (digitalpaymentguru.com)	Easy with strong plugin ecosystem (thefinrate.com)
Security	PCI compliant, fraud tools (doorspell.com)	PCI compliant + advanced fraud detection (Radar) (digitalpaymentguru.com)	PCI DSS compliance & tokenization (blog.rackons.in)

2. Objective: Learn about the significance of headers in API requests and responses

Headers are key–value pairs sent along with an API request or response. They provide **metadata (information about the data)**, not the actual data itself.

They help the client (frontend/app) and server understand **how to process the request and response**.

Significance of Headers in API Requests

1. Authentication & Authorization

Headers often carry security credentials:

```
Authorization: Bearer <token>
```

- ✓ Verifies who the user is
 - ✓ Controls access to protected APIs
-

2. Content Type Declaration

Tells the server what kind of data is being sent:

```
Content-Type: application/json
```

- ✓ Prevents data format confusion
 - ✓ Enables correct parsing
-

3. Accept Response Format

Client specifies what format it expects:

```
Accept: application/json
```

- ✓ Ensures compatibility
-

4. API Versioning

Helps manage different API versions:

```
API-Version: v2
```

- ✓ Allows backward compatibility

5. Caching Control

Improves performance:

Cache-Control: no-cache

✓ Reduces unnecessary server calls

Significance of Headers in API Responses

1. Status Information

Indicates result of request:

HTTP/1.1 200 OK

✓ Success, error, redirect info

2. Response Data Type

Server tells format of returned data:

Content-Type: application/json

3. Security Headers

Protect against attacks:

X-Frame-Options: DENY
Strict-Transport-Security: max-age=31536000

✓ Prevents XSS, clickjacking, MITM attacks

4. CORS Handling

Allows cross-domain access:

Access-Control-Allow-Origin: *

✓ Required for frontend–backend communication

5. Rate Limiting Info

Controls API usage:

X-RateLimit-Limit: 1000

✓ Prevents abuse

Simple Example

Request Header:

```
POST /api/login
Content-Type: application/json
Authorization: Bearer abc123
```

Response Header:

```
200 OK
Content-Type: application/json
Set-Cookie: sessionid=xyz
```

Q1. What are HTTP headers, and how do they facilitate communication between client and server?

HTTP headers are key–value pairs sent along with an HTTP request or response. They carry **metadata (extra information)** about the request or response, not the actual body data.

They help the **client (browser/app)** and the **server** understand:

- Who is making the request
- What data format is used
- How the connection should be handled
- What security rules apply

How HTTP Headers Facilitate Client–Server Communication

1. Define Data Format

Tell how data is sent and received:

```
Content-Type: application/json  
Accept: application/json
```

→ Ensures both sides interpret the data correctly.

2. Enable Authentication & Authorization

Used to send credentials:

```
Authorization: Bearer token123
```

→ Server verifies identity and permissions.

3. Control Caching & Performance

Improve speed and reduce server load:

```
Cache-Control: max-age=3600
```

→ Client can reuse cached responses.

4. Maintain Sessions & State

Using cookies:

```
Set-Cookie: session_id=abc123
```

→ Helps track logged-in users.

5. Support Security

Protect against attacks:

```
Strict-Transport-Security  
X-Content-Type-Options
```

→ Prevents data tampering and common vulnerabilities.

6. Enable Cross-Origin Requests (CORS)

Allow frontend and backend on different domains to communicate:

```
Access-Control-Allow-Origin: *
```

7. Provide Request/Response Status

Server tells the result:

```
HTTP/1.1 200 OK
HTTP/1.1 404 Not Found
```

Q2. Describe how to set custom headers in an API request.

Custom headers are user-defined key–value pairs added to an API request to send extra information such as:

- API keys
- Tokens
- App version
- Client ID
- Any business-specific data

Example:

```
X-Client-ID: web_123
X-App-Version: 1.0.0
```

General Format

```
Header-Name: value
```

1. Using JavaScript (Fetch API)

```
fetch("https://api.example.com/users", {
  method: "GET",
  headers: {
    "Content-Type": "application/json",
    "Authorization": "Bearer abc123",
    "X-Client-ID": "web_123"
  }
});
```

2. Using JavaScript (Axios)

```
axios.get("https://api.example.com/users", {
  headers: {
    "Authorization": "Bearer abc123",
    "X-App-Version": "1.0.0"
  }
});
```

3. Using PHP (cURL)

```
$ch = curl_init("https://api.example.com/users");

$headers = [
    "Content-Type: application/json",
    "Authorization: Bearer abc123",
    "X-Client-ID: web_123"
];

curl_setopt($ch, CURLOPT_HTTPHEADER, $headers);
curl_setopt($ch, CURLOPT_RETURNTRANSFER, true);

$response = curl_exec($ch);
curl_close($ch);
```

4. Using Postman

1. Open request
2. Go to **Headers** tab
3. Add:
 - Key: X-Client-ID
 - Value: web_123
4. Send request

5. Using Laravel HTTP Client

```
use Illuminate\Support\Facades\Http;

$response = Http::withHeaders([
    'Authorization' => 'Bearer abc123',
    'X-App-Version' => '1.0.0',
])->get('https://api.example.com/users');
```

3. Objective: Understand the process of uploading images through an API.

It means sending an image file from a client (browser/mobile app/Postman) to a server using an HTTP request (usually **POST** or **PUT**), where the server:

1. Receives the file
2. Validates it
3. Stores it (disk or cloud)
4. Returns a response (URL, ID, or success message)

Overall Process (Step by Step)

1. Client selects an image

From file input, camera, or gallery.

2. Client sends HTTP request

Uses:

- Method: `POST`
- Content-Type: `multipart/form-data`
- Body: image file + optional fields (user_id, title, etc.)

Example header:

- `Content-Type: multipart/form-data`

3. API receives the request

Server reads the uploaded file from request data.

4. Validate the image

Server checks:

- File type (jpg, png, webp)
- File size
- MIME type
- Image dimensions (optional)

5. Store the image

Options:

- Local server folder (e.g., /uploads/)
- Cloud storage (AWS S3, Cloudinary, Firebase)

6. Save reference in database (optional)

Store file name or URL with user record.

7. Send response to client

Example:

```
{
  "status": "success",
  "image_url": "https://example.com/uploads/profile123.jpg"
}
```

Client (Web/Mobile)

|

| POST image (multipart/form-data)

v

API Server

|

Validate Image

|

Store Image

|

Send JSON Response

Q1. What are the common file formats for images that can be uploaded via API?

1. JPEG / JPG

- Extension: .jpg, .jpeg
- Best for: Photographs
- Features:
 - Small file size (compressed)
 - Widely supported
- Limitation: Lossy compression (some quality loss)

2. PNG

- Extension: .png
- Best for: Logos, icons, images with transparency
- Features:
 - Lossless compression
 - Supports transparent background

3. GIF

- Extension: .gif
- Best for: Simple animations
- Features:
 - Supports animation
 - Limited to 256 colors

4. WebP

- Extension: .webp
- Best for: Modern web applications
- Features:
 - Smaller size than JPG/PNG
 - Supports transparency & animation
- Limitation: Older systems may not support it

5. BMP

- Extension: .bmp
- Best for: Raw images (rarely used in APIs)
- Limitation: Very large file size

6. TIFF / TIF

- Extension: `.tiff`, `.tif`
- Best for: High-quality professional images
- Limitation: Large file size, limited web use

Q2.Explain the process of handling file uploads securely in a web application

1. Use HTTPS

- Always accept uploads over **HTTPS**
 - Prevents data interception (man-in-the-middle attacks)
-

2. Validate File Type (Don't trust extension only)

✓ Check:

- File extension (`.jpg`, `.png`, `.pdf`)
- MIME type (`image/jpeg`, `image/png`)
- Actual file signature (magic bytes)

Example (concept):

Allowed: `jpg`, `png`, `pdf`

3. Limit File Size

- Set maximum upload size
- Prevents server overload & DoS attacks

Example:

max 2MB

4. Rename Files Before Saving

- Avoid using original file name
- Prevent path traversal & overwrite

Example:

```
user_123_1700000000.jpg
```

5. Store Files Outside Public Directory

- Or restrict execution in upload folder
- Prevents running malicious scripts

Example:

```
/storage/uploads/    (not /public/)
```

6. Disable Script Execution in Upload Folder

Apache:

```
php_flag engine off
```

Nginx:

```
deny all .php;
```

7. Scan for Malware (Optional but recommended)

- Use antivirus tools or cloud scanners
-

8. Use Randomized Folder Structure

- Makes guessing file paths difficult

Example:

```
/uploads/a9f3d2/
```


9. Set Proper File Permissions

- Files: 644
- Folders: 755

Prevents unauthorized access/modification

10. Protect with Authentication & Authorization

- Only logged-in users can upload
- Limit who can upload what

11. Validate Image Content (for images)

- Re-encode image using libraries (GD / ImageMagick)
- Removes embedded malicious code

12. Log & Monitor Upload Activity

- Track IP, user ID, file name, size
- Detect abuse

User uploads file



Validate type & size



Rename file



Store safely



Return response

4. Objective: Differentiate between SOAP and REST API architectures

SOAP vs REST API Architectures

SOAP (Simple Object Access Protocol) and REST (Representational State Transfer) are two different approaches for building web services and APIs.

1. Basic Difference

Feature	SOAP	REST
Full form	Simple Object Access Protocol	Representational State Transfer
Type	Protocol	Architectural style
Data format	XML only	JSON, XML, HTML, Text
Complexity	High	Simple & lightweight

2. Communication & Transport

Aspect	SOAP	REST
Transport protocols	HTTP, SMTP, TCP	Mainly HTTP
Message structure	Strict XML envelope	Normal HTTP request/response

3. Performance

Aspect	SOAP	REST
Speed	Slower (heavy XML)	Faster (lightweight JSON)
Bandwidth usage	High	Low

4. Security

Aspect	SOAP	REST
Security standard	WS-Security	HTTPS, OAuth, JWT
Enterprise support	Very strong	Good but simpler

5. Caching

Feature	SOAP	REST
Caching	Not supported easily	Supported via HTTP

6. Error Handling

Feature	SOAP	REST
Error format	SOAP Faults (standard)	HTTP status codes

7. Use Cases

SOAP is used when	REST is used when
Banking systems	Web & mobile apps
Enterprise systems	Public APIs
High security needed	Fast & scalable services
Transaction-heavy apps	Microservices

8. Example Request

SOAP (XML)

```
<soap:Envelope>  
  <soap:Body>
```

```
<getUser/>
</soap:Body>
</soap:Envelope>
```

REST (JSON)

```
GET /users/1
```

Response:

```
{ "id": 1, "name": "John" }
```

Q1. What are the key characteristics of SOAP APIs?

1. Protocol-Based

- SOAP is a **strict protocol**, not just a design style
 - Follows defined standards and rules
-

2. XML-Based Messaging

- Uses **XML format only** for requests and responses
 - Structured and machine-readable
-

3. Uses SOAP Envelope Structure

Every message contains:

- **Envelope** – root element
 - **Header** – metadata (security, authentication)
 - **Body** – actual request/response data
 - **Fault** – error information
-

4. Transport Independent

- Can work over **HTTP, HTTPS, SMTP, TCP**, etc.
-

5. Built-in Security (WS-Security)

- Supports:
 - Encryption
 - Digital signatures
 - Authentication tokens
-

6. Strong Typing with WSDL

- Uses **WSDL (Web Services Description Language)**
 - Defines:
 - Available methods
 - Parameters
 - Data types
 - Endpoint URL
-

7. ACID Transaction Support

- Supports reliable transactions
 - Suitable for banking and financial systems
-

8. Standardized Error Handling

- Uses **SOAP Faults** for consistent error messages
-

9. Language & Platform Independent

- Works across Java, .NET, PHP, Python, etc.
-

10. Heavy but Reliable

- More bandwidth usage
- Slower than REST
- Very reliable for enterprise systems

Q2. Describe the principles of RESTful API design

Principles of RESTful API Design

REST (Representational State Transfer) is an architectural style for designing scalable and simple web APIs.

1. Client-Server Architecture

- Client and server are independent
 - UI and backend logic are separated
 - Improves scalability and maintainability
-

2. Stateless Communication

- Each request contains all necessary information
- Server does not store client session state

Example:

Authorization: Bearer token

3. Resource-Based URLs

- Everything is a **resource** (users, products, orders)
- Use nouns, not verbs

Good:

/users/10

Bad:

/getUserById

4. Use Standard HTTP Methods

Method	Purpose
GET	Retrieve data

Method	Purpose
POST	Create new resource
PUT/PATCH	Update resource
DELETE	Remove resource

5. Use Proper HTTP Status Codes

Code	Meaning
------	---------

200	OK
201	Created
400	Bad Request
401	Unauthorized
404	Not Found
500	Server Error

6. Representation of Resources

- Data usually in **JSON**
- Sometimes XML

Example:

```
{ "id": 1, "name": "Laptop" }
```

7. Uniform Interface

- Consistent structure
 - Predictable endpoints
 - Standard request/response format
-

8. Caching Support

- Use HTTP caching headers

Cache-Control: max-age=3600

9. Layered System

- API may use proxies, gateways, load balancers
 - Client doesn't need to know
-

10. Versioning

- Maintain backward compatibility

Examples:

/api/v1/users

or

API-Version: 1

11. Security

- Use HTTPS
- Authentication: OAuth, JWT, API keys
- Rate limiting

Q5. Objective: Explore the structure and implementation of a product catalog in an e-commerce system.

1. What is a Product Catalog?

A **product catalog** is a structured collection of all products sold in an e-commerce platform, including:

- Product details (name, price, description)
- Categories
- Images
- Stock information
- Variants (size, color, etc.)

It is the **core data system** that powers browsing, searching, and purchasing.

2. High-Level Structure of a Product Catalog



3. Database Structure (Typical Tables)

1. categories

field	type
id	int
name	varchar
parent_id	int (for subcategories)

2. products

field	type
id	int
category_id	int
name	varchar
description	text
price	decimal
status	active/inactive

3. product_images

| id | product_id | image_url |

4. product_variants

| id | product_id | size | color | price | stock |

5. inventory

| product_id | quantity |

4. Key Features of a Product Catalog

✓ Categories & Subcategories

Helps users browse easily.

✓ Product Details Page

Shows:

- Name
- Images
- Price
- Description
- Ratings

✓ Search & Filtering

Filter by:

- Price
- Brand
- Category
- Rating

✓ Product Variants

Example:

- T-shirt → Size: S/M/L → Color: Red/Blue

✓ Inventory Management

Tracks stock and availability.

5. API Structure for Product Catalog (Example – REST)

Get all products

GET /api/products

Get product by ID

GET /api/products/25

Filter by category

GET /api/products?category=shoes

Add product (admin)

POST /api/products

6. Simple Product JSON Example

```
{
  "id": 101,
  "name": "Running Shoes",
  "price": 2999,
  "category": "Footwear",
  "variants": [
    { "size": "8", "color": "Black", "stock": 10 },
    { "size": "9", "color": "Blue", "stock": 5 }
  ],
  "images": [
    "img1.jpg",
    "img2.jpg"
  ]
}
```

7. Implementation Flow

Admin adds product

↓

Saved in database

↓
API fetches data
↓
Frontend displays catalog
↓
Customer browses & buys

8. Technologies Commonly Used

- Backend: PHP (Laravel), Node.js, Django
 - Database: MySQL, PostgreSQL, MongoDB
 - Search: Elasticsearch
 - Storage: Local / AWS S3
 - Frontend: React, Vue, HTML/CSS
-

9. Why Product Catalog is Important

- ✓ Organizes products
- ✓ Enables fast searching
- ✓ Improves user experience
- ✓ Supports inventory tracking
- ✓ Drives sales

Q1. What are the key components of a product catalog?

Key Components of a Product Catalog

1. Product Information

Basic details of each product:

- Product name
- Description
- SKU / Product ID
- Brand
- Specifications (size, weight, material, etc.)

2. Categories & Subcategories

Organizes products for easy browsing:

- Main categories (Electronics, Clothing)
- Subcategories (Mobiles → Smartphones)

3. Pricing Details

Includes:

- Base price
- Discounts / offers
- Taxes
- Currency

4. Product Images & Media

- Multiple images (front, back, zoom)
- Videos (optional)

5. Product Variants

Different versions of the same product:

- Size
- Color
- Model
- Storage

Example: T-shirt (S, M, L) in Red/Blue

6. Inventory / Stock Information

- Available quantity
- Stock status (In stock / Out of stock)
- Warehouse location (optional)

7. Attributes & Filters

Used for searching and filtering:

- Brand
- Price range
- Rating
- Color
- Size

8. Product Status & Availability

- Active / inactive
- Visible / hidden
- Launch date / expiry date

9. SEO & Metadata

For search engines:

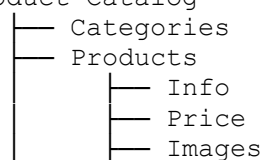
- Product URL (slug)
- Meta title
- Meta description
- Keywords

10. Ratings & Reviews (Optional but important)

- Customer ratings
 - Reviews & comments
-

Simple Structure View

Product Catalog





Q2. How can you ensure that a product catalog is scalable?

Ensuring that a **product catalog is scalable** means it can handle **growth in products, users, traffic, and data** without performance problems. Here are the key strategies:

1. Efficient Database Design

Use normalization + selective denormalization

- Normalize core data (products, categories, variants)
- Denormalize read-heavy fields (price, stock) for faster queries

Proper indexing

- Index:
 - product_id
 - category_id
 - sku
 - price
 - status

2. Use Pagination & Lazy Loading

Instead of loading all products:

```
GET /api/products?page=1&limit=20
```

- ✓ Reduces memory usage
 - ✓ Faster response times
-

3. Implement Caching

Types:

- Database query cache (Redis/Memcached)
- API response cache
- CDN for images

Example:

```
Cache-Control: max-age=3600
```

4. Use Search Engine for Large Catalogs

Integrate:

- Elasticsearch
- OpenSearch
- Algolia

Benefits:

- ✓ Fast search
 - ✓ Advanced filtering
 - ✓ Handles millions of products
-

5. Microservices / Modular Architecture

Separate services:

- Product service
- Inventory service
- Pricing service
- Search service

Improves independent scaling.

6. Optimize Images & Media

- Store on cloud (AWS S3, Cloudinary)

- Use CDN
 - Compress images
 - Multiple sizes (thumbnails)
-

7. Horizontal Scaling

- Use load balancers
 - Multiple API servers
 - Read replicas for database
-

8. Asynchronous Processing

For heavy tasks:

- Bulk imports
- Image processing
- Re-indexing search

Use queues:

- RabbitMQ
 - Redis queues
-

9. API Versioning & Rate Limiting

Prevents breaking clients and system abuse.

10. Monitoring & Performance Testing

Tools:

- New Relic
- Grafana
- JMeter

Track:

- Query time
 - API latency
 - Error rate
-

Simple Architecture View

```
Users
  ↓
Load Balancer
  ↓
API Servers
  ↓
Cache (Redis)
  ↓
Database + Search Engine
  ↓
CDN (Images)
```

6. Objective: Understand the functionality and design of a shopping cart system

1. What is a Shopping Cart System?

A **shopping cart system** is a feature of e-commerce platforms that allows customers to:

- Select products for purchase
- Add, remove, or update product quantities
- Calculate totals, taxes, and discounts
- Proceed to checkout and make payments

It acts as a **temporary storage of user-selected items** before the purchase is completed.

2. Key Functionalities

1. Add to Cart

- Users can select products and add them to the cart
- Includes options like size, color, quantity

2. View / Update Cart

- View items in the cart
- Update quantity or product variants
- Remove unwanted items

3. Price Calculation

- Calculates subtotal
- Adds taxes, shipping, discounts, and promo codes
- Provides total payable amount

4. Persist Cart Data

- Save cart for logged-in users in the database
- Save cart for guests in session or cookies

5. Checkout Integration

- Passes cart data to the checkout and payment system
- Ensures accurate order creation

6. Inventory Check

- Confirms product availability before finalizing purchase
- Prevents over-selling

3. Typical Data Structure

Cart Table

Field	Type	Description
id	int	Cart ID
user_id	int	References logged-in user (nullable for guest)
session_id	varchar	For guest cart persistence
created_at	datetime	Timestamp
updated_at	datetime	Timestamp

Cart_Items Table

Field	Type	Description
id	int	Item ID
cart_id	int	Linked cart
product_id	int	Selected product
variant_id	int	Size/color variant
quantity	int	Number of items
price	decimal	Current product price
total	decimal	price × quantity

4. Workflow of a Shopping Cart System

```
User selects product
↓
Add to cart (session or database)
↓
Update quantity / remove items
↓
Apply coupon / discount
↓
Check inventory & calculate totals
↓
Proceed to checkout → Payment
↓
Order created & cart cleared
```

5. Key Design Considerations

a) Persistence

- **Logged-in users:** Store in database
- **Guests:** Store in session or cookies

b) Scalability

- Optimize cart queries
- Use caching (Redis/Memcached) for frequent access

c) Security

- Validate product IDs and prices server-side
- Prevent tampering with session/cart data

d) User Experience

- Show cart icon with item count
- Real-time updates on quantity/price
- Easy removal or save-for-later feature

e) Integration

- Must integrate seamlessly with:
 - Inventory system
 - Pricing & discount engine
 - Checkout/payment system
-

6. Example JSON Structure of a Cart

```
{
  "cart_id": 101,
  "user_id": 5,
  "items": [
    {
      "product_id": 201,
      "name": "Running Shoes",
      "variant": "Size 9, Red",
      "quantity": 2,
      "price": 2999,
      "total": 5998
    },
    {
      "product_id": 202,
      "name": "Sports Socks",
      "quantity": 3,
      "price": 199,
      "total": 597
    }
  ],
  "subtotal": 6595,
  "discount": 500,
  "tax": 360,
  "total": 6455
}
```

Q1. What are the essential features of an e-commerce shopping cart?

Essential Features of a Shopping Cart

1. Add to Cart

- Allows users to **select products** and add them to the cart
- Supports **product variants** like size, color, or model

2. View Cart

- Users can **view all selected items** in one place
- Shows **product image, name, price, and quantity**

3. Update Cart

- Allows **editing quantity** of items
- Ability to **remove items**
- Real-time **price recalculation**

4. Price Calculation

- Shows **subtotal, taxes, shipping, discounts, and total**
- Applies **promo codes, coupons, or special offers**

5. Persistent Cart

- Saves cart **for logged-in users** in the database
- Saves **guest cart** in session or cookies

6. Inventory Check

- Checks **product availability** before checkout
- Prevents over-selling and stock issues

7. Checkout Integration

- Transfers cart items to **checkout and payment system**
- Supports multiple **payment methods**

8. Security

- Validates **product IDs and prices** on the server
- Prevents **tampering with cart data**

9. User Experience Enhancements

- **Mini-cart icon** showing number of items
- **Save for later / wishlist** option
- **Estimated shipping & delivery** display

10. Scalability

- Handles **large number of products and users** efficiently
- Can integrate caching for performance

[User selects product]



[Add to Cart]



[View / Update / Remove items]



[Apply discount / Check inventory]



[Checkout → Payment]



[Order Confirmation]

Q2. Discuss the importance of session management in maintaining a shopping cart

1. What is Session Management?

Session management is the process of tracking a user's interactions with a web application over multiple requests.

- Each user is assigned a **unique session ID**.
- Data can be stored **temporarily** (in server memory, database, or cookies) and retrieved across pages.

In the context of e-commerce, **sessions help track a user's shopping cart** until they complete checkout.

2. Why Session Management is Important for a Shopping Cart

a) Maintains Cart Across Pages

- Users can **browse multiple pages** without losing selected items.
- Example: Adding products on the homepage, then checking cart on the product page.

b) Supports Guest Users

- Even without login, **sessions allow a temporary cart** stored in cookies or server memory.
- Enables checkout after adding multiple items.

c) Persists Cart for Logged-in Users

- When users are logged in, session data can be linked to **user accounts**.
- Cart can be saved in **database** for future visits.

d) Prevents Data Loss

- Without session management, **cart items disappear** on page reload or navigation.
- Ensures smooth user experience.

e) Security

- Sessions generate **unique IDs** for each user.
- Helps prevent tampering with cart data.
- Can be combined with **HTTPS** and **secure cookies** to protect sensitive info.

f) Enables Scalability

- Large platforms use **session storage systems** (Redis, Memcached) to handle millions of carts efficiently.

3. Common Ways to Implement Session Management

Type	Description
Server-Side Session	Cart data stored on server, session ID stored in browser cookie
Client-Side Session	Cart stored in browser (localStorage/sessionStorage or cookies)
Database-Persistent Cart	Cart linked to logged-in user in database for long-term persistence

4. Example Workflow

1. User visits website → server creates **session ID**
2. User adds product to cart → stored in **session data**
3. User navigates multiple pages → cart persists
4. User logs in → session cart merges with **user account cart**
5. User checks out → cart cleared after order completion

7. Understand the concept of web services and their applications

A **web service** is a way for two different software applications to communicate with each other over the internet using standard protocols (usually HTTP).

Key Characteristics

- Platform independent (Java, PHP, Python, .NET all can talk)
- Language independent
- Uses standard web protocols (HTTP/HTTPS)

- Data is usually sent in **JSON** or **XML**
- Machine-to-machine communication (not human UI)

Types of Web Services

1. SOAP (Simple Object Access Protocol)

- Uses XML
- Very strict standards
- High security
- Used in banking, enterprise systems

2. REST (Representational State Transfer) – Most popular today

- Uses HTTP methods (GET, POST, PUT, DELETE)
- Lightweight
- Uses JSON mostly
- Easy to build and consume

How Web Services Work (Simple Flow)

1. Client sends a request (browser/app/software)
2. Server processes the request
3. Server sends a response (JSON/XML)
4. Client uses the response

Real-World Applications of Web Services

1. Mobile Applications

- Login/signup
- Fetch user profile
- Show products or posts

Example: Instagram fetching feed data

2. E-commerce Websites

- Product catalog service
- Payment gateway integration
- Order tracking
- Shipping APIs (FedEx, DHL)

3. Payment Systems

- PayPal, Stripe, Razor pay APIs
- Transaction verification
- Refund processing

4. Social Media Integration

- Login with Google/Facebook
- Share posts
- Fetch user info

5. Banking & Finance

- Account balance check
- Fund transfer
- KYC verification
- Stock price updates

6. Weather & Maps

- Google Maps API
- Open Weather API

7. Micro services Architecture

Large systems are broken into small services:

- User service
 - Order service
 - Payment service
- Each communicates using web services.

8. IoT Systems

- Smart home devices sending data to servers
 - Sensors reporting temperature, motion, etc.
-

Advantages

- Easy system integration
- Reusable services
- Scalable
- Secure with HTTPS & tokens
- Faster development

Q1. Define web services and explain how they are used in web applications

A **web service** is a standardized way for different software applications to communicate with each other over the internet using web protocols such as HTTP/HTTPS. It allows systems built with different programming languages and running on different platforms to exchange data and perform operations.

Web services typically use data formats like **JSON** or **XML** to send and receive information.

How Web Services Are Used in Web Applications

Web applications use web services to connect the frontend (user interface) with backend systems and third-party services.

1. Data Retrieval and Display

When a user opens a page, the web app requests data from a web service.

Example:

Fetching product lists from a server to display on an e-commerce website.

2. User Authentication

Web services verify login credentials and return authentication tokens.

Example:

Login with email/password or “Sign in with Google”.

3. Form Submission

User data (registration, feedback, orders) is sent to the server using web services.

4. Payment Processing

Web apps communicate with payment gateways like PayPal or Stripe using web services to process transactions securely.

5. Integration with External Services

Web applications use APIs for:

- Maps (Google Maps)
- Weather information
- SMS and email notifications
- Social media sharing

6. Backend Operations

Tasks like:

- Order processing
 - Inventory updates
 - Report generation
- are handled using web services.

Q2. Discuss the difference between REST ful and SOAP web services

Feature	RESTful Web Services	SOAP Web Services
Full Form	Representational State Transfer	Simple Object Access Protocol
Type	Architectural style	Protocol
Data Format	JSON (mostly), XML, HTML, text	XML only
Communication Protocol	Uses HTTP/HTTPS	Can use HTTP, SMTP, TCP, etc.

Feature	RESTful Web Services	SOAP Web Services
Ease of Use	Simple and lightweight	Complex and heavyweight
Performance	Faster due to small message size	Slower due to large XML messages
Security	Uses HTTPS, OAuth, JWT	Built-in WS-Security
State	Stateless	Can be stateless or stateful
Caching	Supported	Not supported
Standards	No strict standard	Very strict standards
Error Handling	Uses HTTP status codes	Uses SOAP Fault messages
Bandwidth Usage	Low	High
Scalability	Highly scalable	Less scalable
Learning Curve	Easy	Difficult
Best Used For	Web & mobile apps, micro services	Banking, enterprise systems

8 Objective: Familiarize with REST ful principles and best practices for API design

RESTful Principles

REST (Representational State Transfer) is an architectural style for designing scalable web APIs.

1. Client–Server Architecture

- Client and server are separate.
- Frontend handles UI.
- Backend handles data and logic.

Benefit: Independent development and scalability.

2. Stateless Communication

- Each request contains all required information.

- Server does not store client session state.

Example: Authentication token sent in every request.

3. Resource-Based Design

- Everything is a resource (users, products, orders).
- Use nouns, not verbs.

Good: `/users/10`

Bad: `/getUserDetails`

4. Use Standard HTTP Methods

Method	Purpose
GET	Retrieve data
POST	Create data
PUT	Update data
PATCH	Partial update
DELETE	Remove data

5. Uniform Interface

- Consistent URL structure
 - Standard HTTP status codes
 - Predictable responses
-

6. Representation of Resources

- Data returned as JSON (preferred)
- XML optional

7. Cacheable Responses

- Responses should specify if they can be cached.

8. Layered System

- Client doesn't know if it talks to the main server or a proxy.

Best Practices for REST API Design

1. Use Clear and Meaningful URLs

```
/api/v1/products  
/api/v1/products/25
```

2. Version Your API

```
/api/v1/  
/api/v2/
```

3. Use Proper HTTP Status Codes

Code	Meaning
------	---------

200	OK
-----	----

201	Created
-----	---------

400	Bad Request
-----	-------------

401	Unauthorized
-----	--------------

404	Not Found
-----	-----------

500	Server Error
-----	--------------

4. Use JSON Format

```
{  
  "id": 1,  
  "name": "Laptop",  
  "price": 50000  
}
```

5. Implement Authentication & Authorization

- JWT
 - OAuth 2.0
 - API Keys
-

6. Handle Errors Properly

```
{  
  "error": "Invalid email address"  
}
```

7. Use Pagination, Filtering & Sorting

```
/products?page=2&limit=10  
/products?category=electronics&sort=price
```

8. Secure Your API

- HTTPS only
 - Rate limiting
 - Input validation
-

9. Documentation

- Use Swagger / OpenAPI
 - Provide examples
-

10. Keep APIs Backward Compatible

- Don't break existing clients

Q1. Explain the importance of statelessness in RESTful APIs.

In a **stateless RESTful API**, each client request contains **all the information** needed for the server to process it.

The server does **not store any client session data** between requests.

Example:

Every request includes an authentication token instead of relying on a server-side session.

Why Statelessness is Important

1. Improves Scalability

- Any server can handle any request.
- Load balancers can distribute requests easily.
- No need to sync session data between servers.

Result: System can handle more users.

2. Increases Reliability

- If one server fails, another can continue processing requests.
 - No session data is lost.
-

3. Simplifies Server Design

- Server does not manage session storage.
 - Easier to maintain and debug.
-

4. Enhances Performance

- Caching becomes easier.
 - Less server memory usage.
 - Faster request handling.
-

5. Supports Distributed Systems

- Perfect for microservices architecture.
 - Services remain independent.
-

6. Improves Security

- Tokens (JWT/OAuth) are sent with each request.
 - No session hijacking via server-stored sessions.
-

7. Easier API Testing

- Each request is independent.
 - No need to recreate session state.
-

Example Comparison

Stateful (Not RESTful)

Server stores:

User session = logged in

Stateless (RESTful)

Client sends every time:

Authorization: Bearer <token>

Q2. What is resource identification in REST, and why is it important?

In REST, **resource identification** means uniquely identifying each resource (data entity) using a **URI (Uniform Resource Identifier)**.

A resource can be anything such as:

- A user
- A product
- An order
- A file

Example:

```
/api/users/10  
/api/products/25  
/api/orders/2026
```

Each URI points to **one specific resource**.

Why is Resource Identification Important?

1. Enables Clear Communication

Clients and servers know **exactly which resource** is being accessed or modified.

2. Supports REST Principles

REST is resource-based. Proper identification is the foundation of RESTful design.

3. Improves API Readability

Human-readable URLs make APIs easy to understand and use.

Good: /users/5

Bad: /getUser?id=5

4. Enables Proper Use of HTTP Methods

Same URI with different methods:

Method	Action
GET	Retrieve resource
PUT	Update resource
DELETE	Remove resource

5. Helps with Caching

Unique URIs allow browsers and servers to cache responses effectively.

6. Improves Scalability

Stateless servers rely on URIs to process requests independently.

7. Simplifies API Maintenance

Consistent resource paths reduce complexity and errors.

9. Objective: Explore the functionality and usage of the OpenWeatherMap API

OpenWeatherMap is a web service that provides **real-time weather data**, forecasts, historical weather, and air pollution data for any location worldwide.

It is widely used in:

- Web applications
 - Mobile apps
 - IoT devices
 - Dashboards
 - Travel and logistics systems
-

Main Functionalities

1. Current Weather Data

Get live weather information for any city or coordinates.

Includes:

- Temperature
 - Humidity
 - Pressure
 - Wind speed
 - Weather condition (clear, rain, clouds)
-

2. Weather Forecast

Types:

- 3-hour forecast (5 days)
 - Daily forecast (up to 16 days – paid)
-

3. Historical Weather Data

Access past weather information (paid plans).

4. Air Pollution Data

Provides:

- AQI (Air Quality Index)
 - CO, NO₂, PM2.5, PM10 levels
-

5. Geocoding API

Convert:

- City name → latitude & longitude
 - Coordinates → city name
-

6. Weather Maps & Icons

- Weather layers
 - Condition icons
 - Satellite maps
-

How It Works (Basic Flow)

1. Create an account on OpenWeatherMap
 2. Get an **API key**
 3. Send HTTP request with city name or coordinates
 4. Receive response in JSON
 5. Display data in your application
-

Example API Request (Current Weather)

`https://api.openweathermap.org/data/2.5/weather?q=London&appid=YOUR_API_KEY`

Example JSON Response

```
{
  "name": "London",
  "main": {
    "temp": 289.5,
    "humidity": 82
  },
  "weather": [
    { "description": "light rain" }
  ],
  "wind": {
    "speed": 4.1
  }
}
```

Common API Parameters

Parameter	Description
q	City name
lat, lon	Coordinates
appid	API key
units	metric / imperial
lang	Language

Example with metric units:

```
&units=metric
```

Usage in Web Applications

Example: JavaScript (Fetch API)

```
fetch("https://api.openweathermap.org/data/2.5/weather?q=Delhi&units=metric&appid=YOUR_API_KEY")
  .then(res => res.json())
  .then(data => {
    console.log("Temperature:", data.main.temp);
    console.log("Condition:", data.weather[0].description);
  });
```

Best Practices

- Keep API key secret (use environment variables)
 - Use HTTPS
 - Cache responses
 - Handle errors (invalid city, limit exceeded)
 - Respect rate limits
 - Use metric units for consistency
-

Advantages

- Easy to use

- Global coverage
 - Free tier available
 - JSON format
 - Reliable and fast
-

Limitations

- API call limits in free plan
- Some features are paid
- Occasional data delays

Q2.Explain how to authenticate and make requests to the OpenWeatherMap API.

1. Authentication in OpenWeatherMap API

OpenWeatherMap uses **API key–based authentication**.

Step 1: Create an Account

Go to the OpenWeatherMap website and sign up.

Step 2: Get Your API Key

After login:

- Open **API keys** section in your dashboard
- Copy your key (example: abc123xyz)

This key uniquely identifies your application.

2. How Authentication Works

You include the API key in every request using the parameter:

```
appid=YOUR_API_KEY
```

3. Making an API Request (Basic Example)

Current weather by city:

```
https://api.openweathermap.org/data/2.5/weather?q=Delhi&appid=YOUR_API_KEY
```

With metric units:

```
https://api.openweathermap.org/data/2.5/weather?q=Delhi&units=metric&appid=YOUR_API_KEY
```

4. Example Request Using JavaScript

```
const apiKey = "YOUR_API_KEY";
const url =
`https://api.openweathermap.org/data/2.5/weather?q=Mumbai&units=metric&appid=${apiKey}`;

fetch(url)
  .then(response => response.json())
  .then(data => {
    console.log("Temperature:", data.main.temp);
    console.log("Weather:", data.weather[0].description);
  })
  .catch(error => console.log("Error:", error));
```

5. Example Request Using cURL

```
curl
"https://api.openweathermap.org/data/2.5/weather?q=London&appid=YOUR_API_KEY"
```

6. Using Coordinates Instead of City Name

```
https://api.openweathermap.org/data/2.5/weather?lat=28.61&lon=77.20&appid=YOUR_API_KEY
```

7. Handling Errors

Common error responses:

Code	Meaning
401	Invalid API key
404	City not found
429	Too many requests

Code	Meaning
------	---------

500	Server error
-----	--------------

Example error JSON:

```
{
  "cod": 401,
  "message": "Invalid API key"
}
```

8. Security Best Practices

Do:

- Store API key in environment variables
- Use HTTPS
- Restrict key usage (if available)
- Cache responses
- Handle rate limits

Avoid:

- Hard-coding keys in frontend code
 - Sharing keys publicly
 - Committing keys to GitHub
-

9. Authentication Summary (Short Note)

OpenWeatherMap API uses API key authentication. Developers must register to obtain a key and include it as the `appid` parameter in each HTTP request to access weather data securely.

10. Objective: Understand the use of Google Maps Geocoding API for location services

Google Maps Geocoding API – Overview

The **Geocoding API** is a Google Maps web service that converts:

- **Addresses** → **Geographic coordinates (latitude, longitude)**
- **Coordinates** → **Human-readable addresses**

This enables applications to work with real-world locations.

What is Geocoding?

Type	Description
Geocoding	Address → Latitude & Longitude
Reverse Geocoding	Latitude & Longitude → Address

Why It Is Important for Location Services

Modern apps need to:

- Show locations on maps
- Track deliveries
- Find nearby places
- Auto-fill addresses
- Calculate distance & routes

The Geocoding API makes all this possible.

Main Uses of Google Maps Geocoding API

1. Convert Address to Coordinates (Geocoding)

Example:

"Connaught Place, Delhi"
→ 28.6315, 77.2167

Used for:

- Placing markers on maps
- Distance calculation
- Navigation

2. Convert Coordinates to Address (Reverse Geocoding)

Example:

28.6315, 77.2167
→ Connaught Place, New Delhi, India

Used for:

- Showing user location
- Delivery tracking
- Ride-sharing apps

3. Address Validation

- Ensures addresses are real and formatted correctly
- Reduces delivery errors

4. Auto-Complete & Location Search

Used with Google Places API to:

- Suggest locations while typing
- Improve user experience

5. Location-Based Services

- Nearby shops
 - Hospitals
 - ATMs
 - Restaurants
-

How It Works (Simple Flow)

1. User enters address OR app gets GPS coordinates
 2. App sends request to Geocoding API
 3. Google returns JSON response
 4. App uses the location data
-

Example API Requests

Geocoding (Address → Coordinates)

`https://maps.googleapis.com/maps/api/geocode/json?address=Delhi&key=YOUR_API_KEY`

Reverse Geocoding (Coordinates → Address)

`https://maps.googleapis.com/maps/api/geocode/json?latlng=28.61,77.20&key=YOUR_API_KEY`

Example Response Data

- Latitude & Longitude
- Formatted address
- City, state, country
- Postal code
- Place ID

Q1.What is geocoding, and how does it work with the Google Maps API?

Geocoding is the process of converting a human-readable address (such as a city, street, or landmark) into geographic coordinates (**latitude and longitude**).

Example:

"Connaught Place, New Delhi"
→ 28.6315, 77.2167

The reverse process is called **reverse geocoding**:

28.6315, 77.2167
→ Connaught Place, New Delhi, India

How Geocoding Works with the Google Maps API

Google provides the **Geocoding API** as part of Google Maps Platform to perform this conversion.

Step-by-Step Working

1. Get an API Key

- Create a project in Google Cloud Console
 - Enable **Geocoding API**
 - Generate an API key
-

2. Send a Request

Address → Coordinates (Geocoding)

`https://maps.googleapis.com/maps/api/geocode/json?address=Delhi&key=YOUR_API_KEY`

Coordinates → Address (Reverse Geocoding)

`https://maps.googleapis.com/maps/api/geocode/json?latlng=28.61,77.20&key=YOUR_API_KEY`

3. Google Processes the Request

- Matches the address with its location database
 - Finds the best geographic match
-

4. Receive JSON Response

Example fields returned:

```
{
  "results": [
    {
      "formatted_address": "New Delhi, Delhi, India",
      "geometry": {
        "location": {
          "lat": 28.6139,
          "lng": 77.2090
        }
      }
    }
  ]
}
```

```
],  
  "status": "OK"  
}
```

5. Use the Data in Your Application

- Place marker on map
 - Calculate distance
 - Show address
 - Find nearby places
 - Track delivery
-

Why It Is Useful

- Location-based services
- Navigation & routing
- Ride-sharing apps
- Delivery systems
- Real-estate platforms
- Travel apps

Q2. Discuss the potential applications of the Google Maps Geocoding API in web applications

1. Displaying Locations on Maps

Geocoding converts addresses into latitude and longitude so they can be shown on Google Maps.

Example uses:

- Showing store branches
 - Office locations
 - Event venues
 - Hotels or restaurants
-

2. Delivery & Logistics Systems

Used to:

- Validate customer addresses
- Convert delivery address to coordinates
- Track delivery agents in real time
- Calculate distance and ETA

Industries: E-commerce, food delivery, courier services

3. Ride-Sharing & Taxi Applications

- Convert pickup and drop locations into coordinates
- Find nearest drivers
- Calculate routes and fares
- Show live vehicle position

Example: Uber, Ola-like systems

4. Address Auto-Fill & Validation

When users type an address:

- Suggest correct locations
- Auto-complete city, state, country
- Reduce wrong or fake addresses

Used in:

- Registration forms
 - Checkout pages
-

5. Location-Based Search

Find nearby:

- Hospitals

- Restaurants
- ATMs
- Schools
- Hotels

The API converts the user's location into coordinates to perform proximity searches.

6. Real Estate Applications

- Show properties on maps
 - Filter properties by area
 - Calculate distance to schools, offices, metro stations
 - Neighborhood analysis
-

7. Travel & Tourism Websites

- Show tourist attractions
 - Hotel locations
 - Route planning
 - Travel distance calculation
-

8. Emergency & Safety Systems

- Identify exact location from GPS
 - Convert coordinates to readable address
 - Dispatch emergency services quickly
-

9. Business Analytics & Reporting

- Analyze customer locations
 - Identify high-demand areas
 - Optimize warehouse or store placement
-

10. Social Media & Event Platforms

- Tag locations in posts
- Show event venues on maps
- Find nearby events or friends